Lightweight Soundness for Towers of Language Extensions

Alejandro Serrano Jurriaan Hage

Department of Information and Computing Sciences
Utrecht University, The Netherlands
{A.SerranoMena, J.Hage}@uu.nl

Abstract

It is quite natural to define a software language as an extension of a base language. A compiler builder usually prefers to work on a representation in the base language, while programmers prefer to program in the extended language. As we define a language extension, we want to ensure that desugaring it into the base language is provably sound.

We present a lightweight approach to verifying soundness by embedding the base language and its extensions in Haskell. The embedding uses the final tagless style, encoding each language as a type class. As a result, combination and enhancement of language extensions are expressed in a natural way. Soundness of the language extension corresponds to well-typedness of the Haskell terms, so no extra tool but the compiler is needed.

Categories and Subject Descriptors I.2.2 [*Automatic Programming*]: Program transformation; D.3.2 [*Language Classifications*]: Extensible languages

Keywords domain-specific languages, extensible languages, type soundness

1. Introduction

Most programming languages include plenty of *sugar* in their syntax. Sugar adds no new power to the language, but can be understood in terms of more basic constructs. Examples of syntactic sugar include `for-each` loops as found in Java or C#, and the `do` notation used in Haskell for monads.

Some programming languages go a step further and offer facilities to the programmer to add sugar to the language, essentially *extending the language*. Macros in Racket (Tobin-Hochstadt et al. 2011) and Scala (Burmako 2013), quasi-quotes in Haskell (Mainland 2007) and packages in Ciao Prolog (Hermenegildo et al. 2012) are well-known examples. Syntactic extensions are defined by a *translation* of the syntactic sugar into (more) basic constructs. An example is the translation of **let** blocks of the form **let** $x = e$ **in** b into the bare λ-calculus, that has only abstraction and application: $(\lambda x \to b)\ e$. A piece of code can be desugared by applying translation rules until only base language constructs are left.

Albeit simple, such a technique has a distinct disadvantage: the code which is input to further stages of the compiler is no longer the same. This has various practical implications. E.g., type errors discovered by analyzing desugared code may be found in code not written by the programmer, leading to cryptic type error messages. For that reason, we prefer the pipeline from (Lorenzen and Erdweg 2013, 2016), in which language extensions define both their translation and their *typing rules*. Programmer code is checked using the base language typing rules plus those of each extension, and only if the code is found to be well-typed, it is translated to the base language. But what if the desugared code has a type different from the original code?

To avoid such a situation, we need to ensure the *soundness* of the typing rule of the language extension with respect to the base language. Quoting (Lorenzen and Erdweg 2016), we want the following to hold:

> For any base language B, extension X and program p, if p is well-typed in $B \cup X$ and p desugars to p', then p' is well-typed in B.

Furthermore, we want to be able to check this property in a modular and compositional way. That is, we want to have a procedure which guarantees that once a language extension is checked, its new typing rules can be combined without conflicts with the base language; and also that different language extensions can be used without having to do extra work to check the resulting combination.

In this paper we present a lightweight methodology to check the soundness of language extensions using a tagless final encoding (Carette et al. 2009):

- We represent language extensions using Haskell type classes, in such a way that well-typedness implies that the typing rule and translation are sound with respect to a base language. This representation lends itself to easy combination of extensions.
- We rework this technique to check not only for soundness, but also for completeness with respect to a subset of the base language.

After a simple introductory example that features a functional language, we provide two additional case studies that show the broad applicability of our ideas: the validation of domain specific type rules (Heeren et al. 2003), and the verification of extensions in the context of an object-oriented language, in the vein of (Lorenzen and Erdweg 2016).

Due to a lack of space, we cannot show how we ported our technique with minor variations to the dependently-typed language Agda (the main difference is that Agda does not support type classes, and we have to use parameterized modules instead). However, both implementations are publicly available at `https://git.science.uu.nl/f100183/lightweight-soundness`.

2. Type Level Programming in Haskell

We first provide a short tutorial of type level programming in GHC's dialect of Haskell: in Haskell every value possesses a type, and every

PEPM'17, January 16–17, 2017, Paris, France
© 2017 ACM. 978-1-4503-4721-1/17/01...$15.00
http://dx.doi.org/10.1145/3018882.3018884

type a *kind*. Originally, kinds had a very simple structure: runtime values are given types of kind $*$, whereas type constructors have kinds built with $\rightarrow$.

$$Maybe :: * \rightarrow * \qquad Int :: * \qquad Maybe\ Int :: *$$

This simple picture experienced a radical change with the introduction of data type promotion (Yorgey et al. 2012). When this extension is enabled in GHC, each data type defined in the source is *promoted* one level up: the defined type is promoted into a kind, and each of the data constructors becomes a type constructor. Take the following data type representing unary numbers:

```
data Nat = Z | S Nat
```

Promoting Nat generates two new type constructors:

$$'Z :: 'Nat \qquad 'S :: 'Nat \rightarrow 'Nat$$

Following GHC nomenclature, we prepend the name of promoted types and kinds with a ' sign. Note that in source code files, that sign can be omitted if no confusion arises.

Throughout the paper we shall encounter several uses of promoted lists. For example, $Int\ ':\ Bool\ ':\ '[]$ represents a type level list whose elements are the basic types Int and $Bool$. The kind of such a list, that is, the "type" of such list of types, is $'[*]$.

Promoted data types are very useful when used in conjunction with Generalized Algebraic Data Types (GADTs). Each data constructor of a GADT may return a different instance of the defined data type. In that way, we can *index* a type with some static information. Lists indexed by their length, usually called vectors, are the archetypical example:

```
data Vec n a where
  Nil  ::                    Vec 'Z      a
  Cons :: a → Vec n a → Vec ('S n) a
```

The compiler is able to check and infer some (but not all) indices when using a value of a GADT:

```
> :t Cons True (Cons False Nil)
Cons True (Cons False Nil) :: Vec ('S ('S 'Z)) Bool
```

In contrast with dependently-typed languages, in Haskell we can only index by types. Data type promotion gives the illusion that we are sharing elements between the value and type worlds, but in reality they are strictly separated.

Type families (Schrijvers et al. 2007) play the role of functions over types, although operationally they work via term rewriting. Type families can be either open, that is, allow the definition of cases in different places; or closed, in which case all rules must be stated right after the declaration. We only make use of the latter in this paper. As an example, here is how one defines type level addition over promoted Nats:

```
type family Add m n where
  Add 'Z     n = n
  Add ('S m) n = 'S (Add m n)
```

Once we can perform addition at the type level, we can give a precise type to the operation of appending two Vecs.

```
append :: Vec m a → Vec n a → Vec (Add m n) a
append Nil         ys = ys
append (Cons x xs) ys = Cons x (append xs ys)
```

Note that this code typechecks because both Add and $append$ have similar recursion schemes. Had we defined one of the two differently, the other should be changed to accommodate the new scheme.

In Haskell, types can contain *constraints*, like $Eq\ a$ in the type for $Eq\ a \Rightarrow a \rightarrow a \rightarrow Bool$ that says that this type only holds for types a that belong to the Eq type class. Another common constraint is that of type equalities, such as $a \sim Bool$. When the appropiate extension (Bolingbroke 2011) is enabled, constraints are treated as inhabitants of a new kind, $Constraint$, meaning that type level programming can also be applied to constraints. For example, we can define a synonym for a set of constraints using the same aliasing procedure that works for regular types,

```
type ReadShow a = (Read a, Show a)
```

Type families may also consume or produce constraints. The following family takes a parametric constraint c, such as $Show$ or Eq, and produces a set of constraints by applying c to each type in a type-level list. Note that GHC considers tupling of constraints as their conjunction; thus the level of nesting is irrelevant.

```
type family All c xs where
  All c '[]       = ()
  All c (x ': xs) = (c x, All c xs)
```

We can now write $All\ Show\ '[Int, Bool]$, and it is equivalent to $(Show\ Int, Show\ Bool)$.

2.1 Proxy Values

Let us consider this archetypical example of ambiguity in Haskell:

```
ambiguous = show . read
```

The type of the entire expression is $ambiguous :: String \rightarrow String$: we serialize the string into some type τ and then serialize it back into a string. The problem is that the code as it stands does not give enough information to fix τ, and since the type does not appear in the signature, we cannot give it externally at call time.

In case τ is some fixed type, we can annotate one of the functions with the expected type:

```
notAmbiguous = show . (read :: String → Int)
```

Having to add annotations to fix type variables becomes cumbersome very soon. One of the reasons is that when you specify a type signature, you need to specify *all* the type parameters. However, many of them could be inferred by the compiler; we are only interested in informing the compiler about those that cannot. One solution is to use a *partial type signature* (Winant et al. 2014). Another solution is to *reify* the type information as a value. In order to do so, we introduce a dummy $Proxy$ type:

```
data Proxy a = Proxy
```

We use $Proxy$ in those places in which we need type information from the user. The $Proxy$ only exists to fix a type, we do not even have to match on their value:

```
read' :: Proxy a → String → a
read' _ = read
```

Now we just need to have a type signature *on the Proxy value*:

```
notAmbiguous = show . read (Proxy :: Proxy Int)
```

In this case, the code is bigger than when using an annotation. But as types grow bigger, $Proxy$s give a nice reduction in code size. Given the ubiquity of $Proxy :: Proxy\ a$ terms in the rest of our paper, we shall shorten it to $\mathfrak{p}\langle a \rangle$.

3. Languages as Type Classes

We describe in this section how a base language can be encoded in tagless final style (Carette et al. 2009), and how it can be extended in one or several ways. As we discussed in the introduction, these language extensions are defined by a typing rule and a translation into more basic constructs.

As running example we use the simply-typed λ-calculus, whose typing judgement is given in Figure 1. Note that the typing judge-

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{VAR} \qquad \frac{x : \tau, \Gamma \vdash e : \rho}{\Gamma \vdash \lambda x.\, e : \tau \to \rho}\ \text{ABS}$$

$$\frac{\Gamma \vdash f : \tau \to \rho \qquad \Gamma \vdash e : \tau}{\Gamma \vdash f\ e : \rho}\ \text{APP}$$

Figure 1. Typing rules for simply-typed λ-calculus

```
type family Ix (n :: Nat) xs where
  Ix Z     (x : xs) = x
  Ix (S n) (x : xs) = Ix n xs
```

Figure 2. Lookup by index in a type-level list

ment is syntax-directed: there is only one applicable rule per language construct. This is an important prerequisite for our technique: it gives us a correspondence between the structure of a term and its typing derivation.

For each expression in our language, expressed as a Haskell runtime value, we assign its typing derivation, including information about the environment. Intuitively, we just move things around a bit in the typing judgement, and instead of $\Gamma \vdash e : \tau$, we write $e : WT\ \Gamma\ \tau$ (*WT* stands for *w*ell-*t*ypedness). The next question is how the types in our language are represented. Instead of using a separate universe, we can simply reuse Haskell types. That is, τ must have kind $*$.

Now that we have settled on a representation of types, we turn to the encoding of environments. Instead of a mapping from names to types, we use a nameless representation using de Bruijn indices. An environment is then just a list of types, corresponding to increasing de Bruijn indices. Putting everything together, the kind of *WT* is:

```
WT :: [*] ->    -- The environment
      *   ->    -- The assigned type
      *
```

For example, the term $\lambda x \to x$ can be given any type $\alpha \to \alpha$ in any possible environment. We represent this fact as $\underline{\lambda\, x \to x} :: forall\ \Gamma\ \alpha\,.\,WT\ \Gamma\ (\alpha \to \alpha)$, where $\underline{\lambda\, x \to x}$ is the yet-to-be-defined representation of the given λ-term.

3.1 Initial Implementation

A first approach to describe λ-terms is to define $\vdash$ as a data type. Each constructor represents a typing rule, which are in bijection with the terms, due to the syntax-directedness of the type system. We can do so in Haskell using GADTs:

```
data WT Γ τ where
  Var :: Proxy i                        -> WT Γ (Ix i Γ)
  App :: WT Γ (α -> β) -> WT Γ α        -> WT Γ β
  Abs :: WT (α : Γ) β                   -> WT Γ (α -> β)
```

The types of the constructors are almost direct translations of the typing rules of Figure 1. The exception is *Var*, which needs to look for the variable in the environment. Since we have settled for a de Bruijn representation of environments, we refer to the variable we want by position. The type family *Ix*, defined in Figure 2, looks up a given position in a type-level list. Since there is no parameter associated with the index *i*, we need a *Proxy* value to communicate it to the compiler, as described in § 2.1.

The identity function can be typed using this encoding:

```
Abs (Var p⟨'Z⟩) :: forall Γ α . WT Γ (α -> α)
```

Because of our use of de Bruijn indices, the binding structure is not apparent in the term. In § 6 we discuss another alternative which reuses Haskell binding forms inside our own language. Well-scoping is checked: had we referenced the variable '*S* '*Z* instead of '*Z*, the type would have reflected that at least one type must inhabit the environment.

```
Abs (Var p⟨'S 'Z⟩) :: forall Γ α β . WT (β : Γ) (α -> β)
```

The encoding using a data type (usually referred to as initial encoding) has a big disadvantage: it is impossible to add new constructors without changing the source code. This means that there is no way to represent our language extensions using the same encoding. Furthermore, we cannot mix two different base languages when describing an extension.

This problem is an instance of the most general Expression Problem. We employ the solution proposed by (Carette et al. 2009): instead of using a data type, our language is defined via a *type class*. The types corresponding to each rule do not change, but they are no longer constructors:

```
class Base wt where
  var :: Proxy i                        -> wt Γ (Ix i Γ)
  app :: wt Γ (α -> β) -> wt Γ α        -> wt Γ β
  abs :: wt (α : Γ) β                   -> wt Γ (α -> β)
```

From the concrete syntax perspective, the only change compared to the previous encoding is the different capitalization on *wt* imposed by Haskell's syntax. The types assigned to expressions are only slightly different, mentioning the class *Base* we have defined.

```
abs (var p⟨'Z⟩) :: Base wt => wt Γ (α -> α)
```

During the rest of the paper we extend this *Base* type class in order to introduce new constructs in the described language.

3.2 Two Flavors of let

Our first addition is the translation of **let** $x = e$ **in** b blocks to $(\lambda x \to b)\ e$: for every new construct we introduce we must also provide a new typing rule, which is checked for soundness with respect to the translation. For **let** bindings, the rule reads:

$$\frac{\Gamma \vdash e : \tau \qquad x : \tau, \Gamma \vdash b : \rho}{\Gamma \vdash \textbf{let}\ x = e\ \textbf{in}\ b : \rho}\ \text{LET}$$

As we did for *Base*, we can define a type class which encodes the typing rule of **let**.

```
class Let wt where
  let_ :: wt Γ τ -> wt (τ : Γ) ρ -> wt Γ ρ
```

However, this language is not so useful because it has only one construct, namely **let** bindings. What we really want is an *extension* of *Base*, which we achieve by letting the *Let* type class have *Base* as a *superclass*. The translation from the extension to the base language is given as a *default definition*. The well-typedness of such a definition ensures that the translation is itself well-typed, because we have encoded the typing rules of λ-calculus in the signatures of *Base*. Furthermore, by using default definitions we ensure that we only use constructs from either the base language or the extension being defined.

```
class Base wt => Let wt where
  let_ :: wt Γ τ -> wt (τ : Γ) ρ -> wt Γ ρ
  let_ e b = app (abs b) e
```

If the translation does not comply with its typing obligations, a compile error is issued. For example, if we mistakenly write *app* (*abs e*) *b*, GHC shows the following error message:

```
* Couldn't match type 'gamma' with 'tau : gamma'
  'gamma' is a rigid type variable bound by
  the type signature for:
```

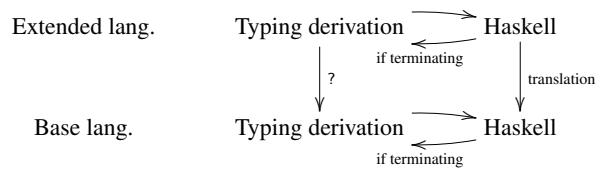

Figure 3. Transformations between languages

```
    let_ :: forall (gamma :: [*]) tau rho.
            wt gamma tau -> wt (tau : gamma) rho
            -> wt gamma rho
  Expected type: wt gamma rho
    Actual type: wt (tau : gamma) rho
* In the expression: app (abs e) b
  In an equation for 'let_': let_ e b = app (abs e) b
```

This tells us that we are using an expresion in environment Γ while we postulated it to live in a larger one $\tau : \Gamma$.

The diagram in Figure 3 shows the steps involved in our soundness check: we want to translate typing derivations for the extended language to derivations in the base language (the arrow marked with a question mark). Our Haskell data types represent the typing derivations for each language, and we define the desired translation between these Haskell data types. Since we can go back and forth between Haskell data types and typing derivations, this implements the desired arrow. A formal definition is given in § 3.4.

We do have to be careful here because of the limitations of Haskell as a language for proofs: since the compiler does not check for termination, we can type expressions which do not correspond to finite typing derivations. The soundness check depends on this, but the Haskell compiler cannot prove it. To overcome this limitation, one could apply our technique in a language like Agda, which can guarantee the termination of functions. In fact, we have done so, but due to lack of space cannot provide any details.

We are not limited to extending the base language once: we can build whole towers of language extensions! These towers need not be linear: a certain extension E may use extensions A and B in its translation. For example, instead of having to introduce one local binding at a time, we offer a new construct,

$$\textbf{let } x_1 = e_1, x_2 = e_2, \ldots \textbf{ in } b$$

in which each binding is visible to the following ones. That is, at e_n we can use all the names $x_1, \ldots, x_{n-1}$. The typing rules for this sort of multiple **let** binding are:

$$\frac{\Gamma \vdash b : \rho}{\Gamma \vdash \textbf{let} - \textbf{in } b : \rho}\ \text{END}$$

$$\frac{\Gamma \vdash e : \tau \qquad x : \tau, \Gamma \vdash \textbf{let } r \textbf{ in } b : \rho}{\Gamma \vdash \textbf{let } x = e, r \textbf{ in } b : \rho}\ \text{MORE}$$

where the translation is just a nesting of simple **let**s. Thus, the extension is based this time not on *Base*, but on the *Let* language we just defined. In the code this fact is reflected on the choice of superclass for *MultiLet*.

```
class Let wt => MultiLet wt where
  end      :: wt Γ ρ → wt Γ ρ
  end   b  = b
  more     :: wt Γ τ → wt (τ : Γ) ρ → wt Γ ρ
  more d ds = let_ d ds
```

In conclusion, using type classes to encode our language gives a natural way to express extensions, via the superclassing mechanism.

```
Lambda.$dmlet_ =
  \ (@ (wt :: [*] -> * -> *)) ($dLet :: Let wt)
    (@ (gamma :: [*])) (@ tau) (@ rho)
    (e :: wt gamma tau)
    (b :: wt (tau : gamma) rho) ->
    let { $dBase = Lambda.$p1Let @ wt $dLet } in
    app @ wt $dBase @ gamma @ tau @ rho
        (abs @ wt $dBase @ tau @ gamma @ rho b)
        e
```

Figure 4. Translation of *let_* into System FC

3.3 Extracting Typing Derivations

Our technique is able to verify the soundness of a typing rule with respect to a translation. However, everything is very implicit: we know that a typing derivation *exists*, but we do not know the derivation *itself*. Knowing how typing derivations are transformed allows us to obtain a complete derivation in the base language. Furthermore, this derivation may affect code generation.

Luckily, GHC can provide us with this information. GHC translates Haskell code, where type abstraction, type application and type class dictionaries are left implicit, into a fully-explicit polymorphic λ-calculus with data types and coercions called System FC. For example, the simple code *show True* leads to an implicit instantiation of the function $show :: Show\ a \Rightarrow a \to String$ with the type *Bool*, and a dictionary describing how to show Boolean values. Once we arrive at the System FC representation[1]:

```
show Bool dShowBool True
```

the dictionary for the *Show Bool* instance, called *dShowBool*, is explicitly passed as an argument to *show*.

Moving back to our language extensions, the translation we write where typing is implicit is elaborated into a similar fully explicit form. Let us take as example the code generated for *let_*, given in Figure 4. We have a first block, of two lines, where those elements which are implicitly quantified in the type signature, like τ, ρ and the type class dictionary (prefixed by $), are abstracted explicitly. Then, we find the parameters we wrote in the definition. The important part for us is how in the code the type parameters to *app* and *abs* are explicitly applied. In particular, it tells us that a derivation

$$\frac{\Gamma \vdash e : \tau \qquad x : \tau, \Gamma \vdash b : \rho}{\Gamma \vdash \textbf{let } x = e \textbf{ in } b : \rho}\ \text{LET}$$

is to be transformed into the following one using only basic constructs from the λ-calculus:

$$\frac{\dfrac{x : \tau, \Gamma \vdash b : \rho}{\Gamma \vdash \lambda x \to b : \tau \to \rho}\ \text{ABS} \qquad \Gamma \vdash e : \tau}{\Gamma \vdash (\lambda x \to b)\ e : \rho}\ \text{APP}$$

In conclusion, from this technique we not only check soundness of typing and translations, but also get a description of the transformation of derivations totally for free.

3.4 Formal Definitions

Up to now, we have worked with the Haskell representation of our language, but have not yet defined formally the concept of *language*, *extension* and *soundness*.

DEFINITION 3.1. *A* language *is an inductively defined set* L.

DEFINITION 3.2. *A* typing $\mathcal{T}$ *from expressions in* L *to types* Σ *depending on contextual information* Γ *is a relation* $\mathcal{T} \subset L \times \Gamma \times \Sigma$. *Given a typing* $\mathcal{T}$*, we call every element* $\Delta \in \mathcal{T}$ *a* typing derivation.

[1] Also called GHC Core.

DEFINITION 3.3. *An expression* $e \in L$ *is* well-typed *in context* Γ *if there exists a typing derivation* $\langle e, \Gamma, \tau \rangle \in \mathcal{T}$.

DEFINITION 3.4. *A typing* $\mathcal{T}$ *is* syntax-directed *if for every well-typed term* $e \in L$ *there exists* only one *corresponding typing derivation.*

In a syntax-directed setting, well-typed expressions and typing derivations form a bijection. As a consequence, when we define our data type of typing derivations in Haskell, we are implicitly defining the language of well-typed expressions in the language.

DEFINITION 3.5. *Consider two syntax-directed typings* $\mathcal{T}$ *and* $\mathcal{T}'$ *corresponding to languages* L *and* L'. *We say that* $\mathcal{T}'$ *is an* extension *of* $\mathcal{T}$ *if there exists an injection* $\iota : \mathcal{T}' \hookrightarrow \mathcal{T}$.

Our definition of languages using type classes ensures that this injection always exists. In Haskell, the type every expression operating in a certain type can be weakened to a type using one of their subclasses. For example,

$$(\equiv) :: \mathit{Eq}\ a \Rightarrow a \to a \to \mathit{Bool}$$

can be given the stricter type

$$(\equiv) :: \mathit{Ord}\ a \Rightarrow a \to a \to \mathit{Bool}$$

DEFINITION 3.6. *Let* L *be a language and* L' *an extension of it. A* translation *from* L' *to* L *is a function* $\delta : L' \to L$ *such that:*

$$\delta \circ \iota = \mathit{id}$$

In other words, expressions from the base language are left unchanged by the translation.
Given corresponding typings $\mathcal{T}$ *and* $\mathcal{T}'$ *for* L *and* L' *we say that a translation* δ *is* sound *if there exists a function over typing derivations* $\hat{\delta} : \mathcal{T}' \to \mathcal{T}$ *which coincides with* δ *in the expression part of the relation.*

This notion of sound translation formalizes the intuitive description given in the introduction. Since the source and target of $\hat{\delta}$ are typing derivations, we know that only well-typed terms are mapped and that well-typed terms are produced as a result.

In our Haskell code, we define $\hat{\delta}$ directly using the default definitions from the type class. Thus, the fact that there exists a corresponding translation on bare expressions holds by construction.

4. Named Functions

Up to now, our translations were always in terms of concepts of the extension and the extended language. In the translation of some constructs, though, some specific functions need to be in place. One archetypical example is **do** notation in Haskell, which is transformed into a series of calls to the $\gg=$ bind operator. We motivate the need for such functions by our first major example: automatically proving the soundness of a domain-specific type rule (Heeren et al. 2003; Serrano and Hage 2016).

4.1 Domain Specific Type Rules

Domain specific type rules provide a mechanism for developers of embedded domain specific languages to give more precise error messages. Such type rules are much like any other, except that they can be annotated with two kinds of extra information:

- A custom error message can be attached to parts of the rule, so that when checking the part fails, the custom message will be displayed instead of the default one.
- The order in which constraints in the type rule are checked is made explicit. This is important because a different ordering of traversal when checking or inferring a type can have an impact on the point in which the error is found.

For example, Figure 5 contains a domain specific type rule for *map*. Constraints aligned vertically should be thought of as checked from top to bottom, so that when the first fails no constraint below it is checked afterwards. Columns of constraints can be checked independently of each other. In the example type rule, this would lead to two errors for the expression *map* 2 3: one for 2 not being a function and one for 3 not being a list.

Domain specific type rules are written by library developers who need not be experts on type checking. For that reason, a check for soundness is desirable. This check should also ensure that after application of a domain specific rule we can get back a derivation using the original rules instead. These two are properties which our technique provides.

The first thing to note is that the ordering and custom messages in domain specific type rules do not play a role in the soundness check. These annotations are only important when an expression is ill-typed, and at this point we are checking the soundness of the domain specific type rule for well-typed expressions. Thus, in the case of *map* the typing rule to check can be simplified to:

$$\frac{\begin{array}{cc}\Gamma \vdash \mathit{fn} : \tau & \Gamma \vdash \mathit{lst} : \rho \\ \tau = \alpha \to \beta & \rho = [\delta] \qquad \alpha = \delta\end{array}}{\Gamma \vdash \mathit{map}\ \mathit{fn}\ \mathit{lst} : [\beta]}\ \text{MAP}^*$$

or even further, if we inline the definitions of the variables:

$$\frac{\Gamma \vdash \mathit{fn} : \alpha \to \beta \qquad \Gamma \vdash \mathit{lst} : [\alpha]}{\Gamma \vdash \mathit{map}\ \mathit{fn}\ \mathit{lst} : [\beta]}\ \text{MAP}$$

We can see this typing rule as an extension of the base λ-calculus, where a custom version of *map* is translated to the original *map* function. Following this line of thought, we define the *MapSpec* type class for *specialized map* application:

```
class Base wt ⇒ MapSpec wt where
  map_ :: wt Γ (α → β) → wt Γ [α] → wt Γ [β]
```

But how do we refer to the *original map* function? None of the basic constructs of λ-calculus is the function *map*. Looking at the environment Γ is not an option either, because the elements there are not named. The solution is to declare a new base language, which extends λ-calculus with a known function *map*:

```
class Base wt ⇒ HasMap wt where
  mapfn :: wt Γ ((α → β) → [α] → [β])
```

and defining *MapSpec* as an extension of *HasMap*:

```
class HasMap wt ⇒ MapSpec wt where
  map_ fn lst = app (app mapfn fn) lst
```

To conclude: our technique is not only useful for defining sound language extensions, but can also be used to soundly change aspects of the typing of a given language.

4.2 List Comprehensions

Now we have the power to refer to functions by name, we can build more substantial extensions. List comprehensions are an example of such an extension. In the Haskell 2010 Report (Marlow et al.) we find the following translation scheme:

> List comprehensions satisfy these identities, which may be used as a translation into the kernel:
>
> ```
> [e | True]      = [e]
> [e | q]         = [e | q, True]
> [e | b, Q]      = if b then [e | Q] else []
> ```

$$\frac{\Gamma \vdash \mathit{fn} : \tau \qquad \Gamma \vdash \mathit{lst} : \rho}{\tau = \alpha \to \beta \text{ otherwise } \texttt{"First arg. is not a fn."} \qquad \rho = [\delta] \text{ otherwise } \texttt{"Second arg. is not a list"} \qquad \alpha = \delta \text{ otherwise } \texttt{"Types of fn. domain and list elt. do not match"}}{\Gamma \vdash \mathit{map}\ \mathit{fn}\ \mathit{lst} : [\beta]}$$

Figure 5. Domain specific type rule for *map fn lst*

$$\begin{aligned}
[e \mid p \leftarrow l, Q] &= \textbf{let } \mathit{ok}\ p = [\,e \mid Q] \\
&\qquad\quad \mathit{ok}\ _ = [\,] \\
&\qquad\quad \textbf{in } \mathit{concatMap}\ \mathit{ok}\ l \\
[e \mid \textbf{let } \mathit{decls}, Q] &= \textbf{let } \mathit{decls} \textbf{ in } [e \mid Q]
\end{aligned}$$

where e ranges over expressions, p over patterns, l over list-valued expressions, b over boolean expressions, *decls* over declaration lists, q over qualifiers, and Q over sequences of qualifiers.

To keep the exposition simple, we restrict to the case in which patterns are simple variables (thus, only $x \leftarrow \mathit{lst}$ is allowed) and all **let** bindings introduce a single declaration.

One difference with the foregoing is that comprehensions are not expressions by themselves, but rather a set of qualifiers and an initial expression. Therefore, we first define a data type which represents the items in a comprehension, called *qualifiers* in the Report.

```
data Qualifier wt Γ Γ' where
  Pat   :: wt Γ [α]   → Qualifier wt Γ (α : Γ)
  Guard :: wt Γ Bool → Qualifier wt Γ Γ
  LetL  :: wt Γ α     → Qualifier wt Γ (α : Γ)
```

Each qualifier is indexed with the way it modifies the environment. That is, a value of type *Qualifier wt* Γ Γ' takes an input environment Γ and changes it to Γ'. In order to describe sequences of qualifiers, we need to use the output environment of a qualifier as the input to the next one, sort of a type-aligned sequence.

```
data QList wt Γ Γ' where
  (:[]:) :: QList wt Γ Γ
  (:/:)  :: Qualifier wt Γ Δ → QList wt Δ Σ
         → QList wt Γ Σ
```

In order to describe a comprehension $[e \mid Q]$, we need both a list of qualifiers Q and the expression e which is executed per item in the comprehension. In contrast to qualifiers, this is an expression, so we can include it as an extension of *Base*.

```
class Base wt ⇒ ListCompr wt where
  compr :: QList wt Γ Γ' → wt Γ' τ → wt Γ [τ]
```

Note that the environment in which e is executed is the final environment Γ' from the sequence of qualifiers.

Looking at the translation of list comprehensions, we need the : and [] constructors to construct a singleton list, the *concatMap* function for the pattern case, and a conditional, called *ifthenelse*, for the case of guards:

```
class Base wt ⇒ HasList wt where
  nilfn  :: wt Γ [α]
  consfn :: wt Γ (α → [α] → [α])
class HasList wt ⇒ HasConcatMap wt where
  cmapfn :: wt Γ ((α → [β]) → [α] → [β])
class Base wt ⇒ IfThenElse wt where
  ifthenelse :: wt Γ Bool → wt Γ τ → wt Γ τ → wt Γ τ
```

Now we can implement the translation from the Report:

```
class (Let wt, HasConcatMap wt, IfThenElse wt)
      ⇒ ListCompr wt where
  compr :: QList wt Γ δ → wt wt τ → wt Γ [τ]
  compr (:[]:)          e
    = app (app consfn e) nilfn
  compr (Pat p   :/: r) e
    = app (app cmapfn (abs (compr r e))) p
  compr (Guard g :/: r) e
    = ifthenelse g (compr r e) nilfn
  compr (LetL l  :/: r) e
    = let_ l (compr r e)
```

This is a perfect example of the ease of building complex towers of extensions using the final tagless style. In this case, *ListCompr* was obtained by a number of different extensions to the base λ-calculus, some of them independent of the others. Finally, note that we never had to give explicit definitions for the constructors and functions, like *consfn* and *ifthenelse*, that we used: we just postulated their existence without giving a translation.

4.3 A Common Interface for Named Functions

Referring to a concrete, named function is such a common task that we can think of giving a general solution. The important thing to notice is that in these examples we do not really care about whether a function is called *concatMap* or *fooBar*: all we care about is its type. Using this idea, we can extend *Base* by means of a new construct which refers to a expression whose type is known.

```
class Base wt ⇒ Known wt where
  ...
  kno :: Proxy τ → wt Γ τ
```

We can give names to some specific types to make things more explicit in translations:

```
mapfn = kno (Proxy :: Proxy ((a → b) → [a] → [b]))
nilfn  = kno (Proxy :: Proxy [a])
consfn = kno (Proxy :: Proxy (a → [a] → [a]))
```

And just use them where previously we had a custom class:

```
class Known wt ⇒ MapSpec wt where
  map_ :: ...
  map_ fn lst = app (app mapfn fn) lst
```

The same scheme can be used even if the function has some qualifiers, like *fmap* with *Functor f*:[2]

```
fmapfn :: (Known wt, Functor f)
       ⇒ wt Γ ((a → b) → f a → f b)
fmapfn = kno Proxy
```

We can use this newly defined *fmapfn* in *MapSpec* instead of *mapfn*. The typing rule is still sound if we do so, because every typing derivation for a list can be converted into one where the typing rules only ask for a *Functor*. We end up with an uneasy feeling,

[2] In this case we can use simply *Proxy* without any type signature, because the type is inferred from the type signature of *fmapfn*.

though: it seems that we should have some notion of "completeness" for this kind of rules. We explore such notion in § 7.

5. Extensibility for Extensions

We have constructed languages in final style in order to make them extensible, and now we want to achieve the same thing for the syntax of comprehensions. Currently, it is still defined via a data type, in initial style. This section is heavily influenced by the techniques found in (Kiselyov 2012).

To illustrate, let us split our qualifiers into two mini-languages: one with patterns and guards, the other contains **let** construct. This means introducing two classes.

```
class Q wt qual | qual → wt where
  pat   :: wt Γ [α]   → qual Γ (α : Γ)
  guard :: wt Γ Bool → qual Γ Γ
class Q wt qual ⇒ QLet wt qual where
  letl  :: wt Γ α     → qual Γ (α : Γ)
```

The translation of **let** bindings to patterns is very direct: $[e \mid \textbf{let}\ x = t, Q]$ is the same as $[e \mid x \leftarrow [t], Q]$, so

```
class (HasList wt, Q wt qual) ⇒ QLet wt qual where
  letl :: wt Γ α → qual Γ (α : Γ)
  letl e = pat (consfn @ e @ nilfn)
```

The question that remains is how to define the following method:

```
class Base wt ⇒ ListCompr wt qual where
  compr :: QList qual Γ Δ → wt Δ τ → wt Γ [τ]
```

In initial style, we define an instance of Q for the previously defined *Qualifier*:

```
instance Q wt (Qualifier wt) where
  pat   = Pat
  guard = Guard
```

and then perform the same translation as before.

This scheme is not very satisfactory, though, because it only allows extending list comprehension syntax with qualifiers which can be translated to Q constructs, which are not very expressive. Instead, what we aim for is to have an extensible *translation from Q to expressions*: the function *compr* itself should be made extensible.

The trick, as discussed in section 2.4 of (Kiselyov 2012) is to create instances of the class which take into account the context in which the operation should work. In our case, if we have a qualifier $Q\ \Gamma\ \Delta$, then the translation should be able to take an expression working on environment Δ (the rest of the sequence of qualifiers), and produce a new expression in environment Γ. This process can be seen as building the expression bottom-up. In attribute grammar terms, this is like computing a synthesized attribute.

```
data LangLst wt elt Γ Δ
   = LangLst {unLangLst :: wt Δ [elt] → wt Γ [elt]}
instance (HasConcatMap wt, IfThenElse wt)
         ⇒ Q wt (LangLst wt τ) where
  pat p   = LangLst $ λrest → app (app cmapfn (abs rest)) p
  guard g = LangLst $ λrest → ifthenelse g rest nilfn
```

Building the comprehension now simply involves calling the functions recursively, starting with the case of an empty list of qualifiers, which just returns the given expression:

```
class HasList wt ⇒ ListCompr wt where
  compr :: QList (LangLst wt elt) Γ δ
        → wt δ elt → wt Γ [elt]
  compr (:[]:)    e = app (app consfn e) nilfn
  compr (q :/: r) e = unLangLst q (compr r e)
```

The extensibility has been moved to the instance of Q for *LangLst*. We are now able to define how *letl* behaves independently of the translation of the other sorts of qualifiers.

```
instance (HasConcatMap wt, IfThenElse wt, Let wt)
         ⇒ QLet wt (LangLst wt τ) where
  letl b = LangLst $ λrest → let_ b rest
```

This example shows that our technique for checking soundness is not restricted to checking only the syntactic sorts already defined in the language. We can add new constructs which do not directly correspond to expressions or statements, and still have our correctness guarantees.

6. De Bruijn versus HOAS

Up to now, all the translations played very well with the environment Γ, but this might not be always the case. Sometimes we need to embed an expression in an environment Γ into a larger environment Δ. Because of the strict rules for environments in our types, this does not come for free: we need a specific combinator for weakening environments. A different approach is to use higher-order abstract syntax (HOAS), in which case we reuse the binding structure of the underlying host language. In this section we look at both approaches and discuss the trade-offs between them.

Consider the following example taken from the section on derived conditionals in Scheme's R^{6}RS (Sperber et al. 2010):

> The `or` keyword could be defined in terms of `if` using `syntax-rules` as follows:
>
> ```
> (define-syntax or
>   (syntax-rules ()
>     ((or) #f)
>     ((or test) test)
>     ((or test1 test2 ...)
>      (let ((x test1))
>        (if x x (or test2 ...))))))
> ```

We want to implement this translation using our technique, adding type checking along the way. We introduce a type class to introduce the necessary Boolean constructors:

```
class Base wt ⇒ HasBool wt where
  true, false :: wt Γ Bool
```

Our first attempt is just a transliteration of the rules:

```
class (Let wt, IfThenElse wt, HasBool wt)
      ⇒ Or wt where
  orlist :: [wt Γ Bool] → wt Γ Bool
  orlist []     = true
  orlist [x]    = x
  orlist (x : xs) = let_ x $
    ifthenelse (var p⟨'Z⟩) (var p⟨'Z⟩) (orlist xs)
```

Unfortunately, that code is not type correct:

```
* Couldn't match type 'gamma' with 'Bool : gamma'
  [ ... ]
  Expected type: wt (Bool : gamma) Bool
    Actual type: wt gamma Bool
```

The problem is that once we are in the body of the **let** binding, the environment has been extended and now has a boolean value as its first element. This means that *xs* cannot longer be used as-is, because the de Bruijn indices point to the wrong location. The solution is to add a new primitive to the *Base* language to *weaken* the environment. A weakened version Γ' of an environment Γ contains all the variables in Γ plus possibly more of them at the beginning.

```
class Base wt where
  ...
  weak :: wt Γ τ → wt (α : Γ) τ
```

Now, we can implement *orlist* by replacing the last recursive application in the last equation, *orlist xs*, with its weakened form, *weak* (*orlist xs*). In that way, the environment of the inner *orlist xs* does not mention *x*.

Another case where weakening is needed is the translation of pairs into its Church encoding, as shown in (Lorenzen and Erdweg 2016). First, let us declare a type synonym for the *Pair* type:

```
type PairT t = (t → t → t) → t
```

If you have a pair, you can obtain each of the projections by applying different functions to the pair itself. Using this idea, we write the first part of our language extension:

```
class Base wt ⇒ Pair wt where
  pair1  :: wt Γ (PairT tat) → wt Γ τ
  pair1 p = app p (abs $ abs $ var ƀ⟨'S 'Z⟩)
  pair2  :: wt Γ (PairT τ) → wt Γ τ
  pair2 p = app p (abs $ abs $ var ƀ⟨'Z⟩)
```

The pair constructor captures the two elements in an abstraction. The issue is that this abstraction pushes a new variable into the environment, which was not available in the Γ for the elements. We solve the problem by weakening:

```
class Base wt ⇒ Pair wt where
  mkPair :: wt Γ τ → wt Γ τ → wt Γ (PairT τ)
  mkPair e1 e2
    = abs $ app (app (var ƀ⟨'Z⟩) (weak e1)) (weak e2)
```

The translation roughly corresponds to transforming (e_1, e_2) into $\lambda s \to s\ e_1\ e_2$ for a fresh variable s.

Having to deal with environments this way is tiresome, though. Once we start dealing with more complex expression manipulation, we might even need to remove elements not at the head of the environment, or to reorder some of its elements. A solution that frees us from this tedious work is *higher-order abstract syntax* (Pfenning and Elliott 1988), usually shortened to HOAS.

With HOAS, we can piggyback on the binding forms of the underlying language (in this case, Haskell) to represent variables and environments. Our values no longer carry a Γ index: Γ comes from the environment in which the expression is valid. Instead of enlarging Γ when a new variable is bound, we use an actual λ-abstraction in which the parameter represents the variable. To define our base language, we can do without the *var* case:

```
class Base_h wt where
  app_h :: wt (α → β) → wt α → wt β
  abs_h :: (wt α → wt β) → wt (α → β)
```

Going back to the Scheme example, the implementations of **let** and the basic Boolean constructors need almost no change:

```
class Base_h wt ⇒ Let_h wt where
  let_h :: wt α → (wt α → wt β) → wt β
  let_h e1 e2 = app_h (abs_h e2) e1
class Base_h wt ⇒ HasBool_h wt where
  true_h, false_h :: wt Bool
  ifthenelse_h  :: wt Bool → wt a → wt a → wt a
```

Our *orlist* implementation is much simpler now:

```
class (Let_h wt, HasBool_h wt) ⇒ Or_h wt where
  orlist_h :: [wt Bool] → wt Bool
  orlist_h []      = true_h
  orlist_h [t]     = t
  orlist_h (t : ts) = let_h t $ λx → ifthenelse_h x x (orlist_h ts)
```

Notice that now we use the binding form λx to introduce the new variable in the **let**. When we need it, we can refer to this binding by name, instead of using a specialized *var* construct with a de Bruijn index. Finally, because weakening is built into Haskell's typing rules, we can call $orlist_h\ ts$ directly.

Given all these advantages, what stops us from using HOAS everywhere? The answer lies in how we want to treat the output of the compiler. As discussed in § 3.3 the elaboration of our translations into a explicitly-typed language such as System FC corresponds to the transformations we need to perform on typing derivations. Derivations are most easily written using a explicit environment, and HOAS makes this part more difficult. Some techniques may help us solve this issue (Atkey et al. 2009), but these remain at this point future work.

7. Checking for Completeness

In § 4.1 we implemented a domain-specific type rule for *map* applied to two arguments:

```
class HasMap wt ⇒ MapSpec wt where
  map_ fn lst = app (app mapfn fn) lst
```

Our translation directly exchanges the *map_* custom syntax with the *map* function. However, as hinted at the end of § 4.3, another translation is possible. In particular, using the *fmap* function which is applicable to any *Functor*, including lists:

```
class Base wt ⇒ HasFmap wt where
  fmapfn :: Functor f ⇒ wt Γ ((α → β) → f α → f β)
class HasFmap wt ⇒ MapSpec wt where
  map_ fn lst = app (app fmapfn fn) lst
```

These two translations, both sound with respect to the underlying language, differ in one important property: the first one is "reversible", whereas the second one is not. That is, given a piece of code, you can replace every expression matching *app* (*app mapfn fn*) *lst* with *map_ fn lst* and the program remains correct. This is not the case for *app* (*app fmapfn fn*) *lst*: the replacement is only possible if *lst* has a list type. If we have a different *Functor*, reversing the translation makes the program ill-typed.

The knowledge of whether a translation is reversible is important for domain-specific type rules (Heeren et al. 2003; Serrano and Hage 2016). Type rules are applied depending only on the *syntactic* structure of a term; in particular, it is not possible to guarantee application of a certain rule only when some variables have a specific type. For that reason, DSL writers must be warned when their custom type rules are not complete. If a rule is both sound and complete, it is indeed a drop-in replacement for its translation.

DEFINITION 7.1. *A sound translation $\delta : L' \to L$ is complete if for every well-typed $e \in L$, every expression $e' \in L'$ such that $\delta(e') = e$ is also well-typed.*

Another way to characterize this notion is by considering the inverse of the translation function[3] operating only on well-typed terms:[4]

$$\sigma : \mathcal{T} \to \mathcal{P}L'$$

A translation is complete if this σ can be extended to produce always well-typed terms, $\hat{\sigma} : \mathcal{T} \to \mathcal{P}\mathcal{T}'$.

[3] Maybe we should call it a *sugar-coater*?

[4] $\mathcal{P}X$ represents the powerset of X.

Once we try to write $\hat{\sigma}$ we ran into a wall. We cannot directly pattern match an expression in the base language, not when the language is expressed in final style. Fortunately, we can take our inspiration from extending extensions, as discussed in § 5.

Consider the original example of the domain-specific type rule for *map fn lst*. First of all, let us settle the types. On the one hand, an expression where λ-terms and the *map* function may appear has type $HasMap\ wt \Rightarrow wt\ \Gamma\ \tau$. On the other hand, a set of expressions where that domain-specific type rule may appear has type $MapSpec\ wt \Rightarrow [wt\ \Gamma\ \tau]$. We want the transformation from the former to the latter to be applicable no matter what *wt* is:[5]

```
unmap_ :: (forall b . HasMap b  ⇒  b Γ τ )
        → (forall m . MapSpec m ⇒ [m Γ τ])
```

The first solution is to have a data type B for which we can write an instance $HasMap\ B$ and at the same time a function $unmap' :: B\ \Gamma\ \tau \to (forall\ m\ .\ MapSpec\ m \Rightarrow [m\ \Gamma\ \tau])$. One simple way is to go back to the initial version of our language,

```
data B Γ τ where
  Var :: Proxy i                  → B Γ (Ix i Γ)
  App :: B Γ (α → β) → B Γ α      → B Γ β
  Abs :: B (α : Γ) β              → B Γ (α → β)
  Map :: B Γ ((α → β) → [α] → [Γ])
```

for which the instances are trivial to write:

```
instance Base B where
  var = Var      app = App      abs = Abs
instance HasMap B where
  map = Map
```

Now we can write the $unmap'$ function by pattern matching on B terms and taking advantage of lists forming a monad:

```
unmap' (App (App Map fn) lst)
  = do fn' ← unmap' fn
       lst' ← unmap' lst
       return (map_ fn' lst')
         ⟨|⟩ return (app (app mapfn fn') lst')
unmap' (Var v)   = return (var v)
unmap' (App f e) = app ⟨$⟩ unmap' f ⟨*⟩ unmap' e
unmap' (Abs e)   = abs ⟨$⟩ unmap' e
unmap' (Map e)   = return mapfn
```

Because of the order in which equations are matched, we know that every time that something with the *map fn lst* shape comes in, the domain-specific type rule applies. For the rest of cases, we need to apply the function recursively.

The second possibility is not to use an initial encoding, but rather to integrate the matching mechanism with the translation to the domain-specific type rule. This involves creating a data type holding a context (Ctx) telling us at which stage of the matching process we are, and that we attach to the expression:

```
data MSC Γ τ = MSC (Ctx Γ τ)
                   (forall m . MapSpec m ⇒ [m Γ τ])
```

There are three cases to consider for this context: whether we have seen the function *map*, whether we have seen it applied to a function, and the case we have not seen *map* at all.

```
data Ctx Γ τ where
  FoundMap    :: Ctx Γ ((α → β) → [α] → [β])
  FoundMapFn  :: (forall m . MapSpec m ⇒ [m Γ (α → β)])
              → Ctx Γ ([α] → [β])
  NotFound    :: Ctx Γ τ
```

[5] The *RankNTypes* extension must be enabled for GHC to accept this code.

The indices in Ctx are quite important. Otherwise, we do not keep enough information for the compiler to know that our code is well-typed. But if we do so, we can write instances for MSC acting as a λ-calculus with *map*:

```
instance Base MSC where
  var v = MSC NotFound $ return (var v)
  app (MSC FoundMap m) (MSC _ fn)
    = MSC (FoundMapFn fn) (app ⟨$⟩ m ⟨*⟩ fn)
  app (MSC (FoundMapFn fn) mfn) (MSC _ lst)
    = MSC NotFound $ do
        fn' ← fn
        lst' ← lst
        return (map_ fn' lst') ⟨|⟩ app (app mapfn fn') lst'
  app (MSC NotFound f) (MSC _ e)
    = MSC NotFound (app ⟨$⟩ f ⟨*⟩ e)
  abs (MSC _ e) = MSC NotFound (abs ⟨$⟩ e)
instance HasMap MSC where
  mapfn = MSC FoundMap (return mapfn)
```

Finally we just need to extract the second part of the MSC value:

```
unmap_ = unmap' where unmap' (MSC _ e) = e
```

Irrespective of the technique used to implement the inverse of the well-typed desugaring, we need to check that the function is indeed an inverse and that the return value covers all possible sugarings of the original expression in the base language. We refrain from doing this for the sake of conciseness. Having to write this inverse explicitly is definitely a disadvantage of this technique: depending on the complexity of the domain-specific type rules, doing so can range from trivial to very difficult.

8. The Java Case Study

We have shown a fair number of extensions to the λ-calculus, but does our technique also work for other languages? In this section we apply our lightweight soundness procedure to define extensions for the `for` loop in the Java language. The same examples are discussed in (Lorenzen and Erdweg 2016), so the reader can compare their technique to ours.

8.1 Modelling the Language

As in the case of the λ-calculus, the first design decision we need to make is the shape of the typing judgements. In this case we have five arguments instead of three:

$$\Gamma; \mathfrak{C} \vdash^{\varsigma} e : \tau$$

This judgement tells us that under environment Γ, inside the scope of a class $\mathfrak{C}$, the construct e is assigned type τ. Java, as many other imperative languages, distinguishes between two kinds of syntactic classes: *expressions* and *statements*; the ς argument in the judgement accounts for that distinction.

Classes are modelled by Haskell types. Usually we represent the kind of Haskell types as $*$, but in order to enhance readability we introduce a synonym for it:[6]

```
type JType = *
```

Java classes are related by *subtyping*: each class may have at most one superclass. In our Haskell world, we use an open type family to encode the direct parent of a class.

```
type family JDirectSuperClass (ty :: JType) :: JType
```

[6] In order to compile the code as is, GHC 8 with the *TypeInType* extension is needed. This type synonym is not essential for running the code, though, and *JType* can be safely replaced by $*$ in all the remaining code.

However, knowing the direct parent of a class is not enough: when type checking we are required to look along the whole chain of superclasses. We define a new constraint *JSuperClass ty p* which holds only if *p* is an ancestor of *ty*.

```
type family JSuperClass ty parent :: Constraint where
  JSuperClass t t = ()
  JSuperClass t p = JSuperClass (JDirectSuperClass t) p
```

Having a type family return *Constraint* as its kind allows us to encode a specific search strategy on superclasses: first look whether the type and the suspected parent are the same, otherwise move one place up. Encoding this same strategy with *JSuperClass* as a type class would have required overlapping instances.

Java is object-oriented: each class comes with a collection of *methods*. Each method is defined by its name and its *signature*, that is, the types of the arguments required and type being returned. As we did with classes, we use the openness of $*$ as a source for method names; to distinguish this usage from names of classes, we introduce a different type synonym *JMethod*. After a class is defined, the methods inside it are set in stone and in principle cannot be extended. For that reason, we have a mapping from each class to the list of defined methods, instead of a relation.

```
type JMethod = *
type JSig    = ([JType], JType)
type family JDirectDefinedMethods (ty :: JType)
            :: '[(JMethod, JSig)]
```

The two additional operations related to methods we need to describe the typing judgement are given in Figure 6. The first one, *JMethodSig*, looks for a method along the inheritance chain. In order to consider all parent methods, we first concatenate all the lists of directly defined method using *JDefinedMethods* and then perform a lookup. Type families which append and lookup in type-level lists are given in Figure 7.

The second operation is checking that a list of types match the signature of a method. In other words, *Matches sig actual* holds when each element in a list *actual* is a subtype of the corresponding element in *sig*. As in the case of the superclass relation, we use a type family returning a *Constraint* in order to implement a custom search strategy. Note that conjoining constraints is modelled by tupling (see, e.g., the last equation of *Matches*).

The type class that defines our language is in Figure 8. The judgements for statements are divided into two parts: *varDecl*, *assign*, *while*, *if_* and *bracket* that represent particular statements and the remainder of the sequence, and *end* and *return_* that signal the end of a sequence of statements. Information, like the declaration of a variable, is propagated along the sequence of statements. The distinction between *end* and *return_* is that the former does not return any value. Brackets can be used to define scopes to prevent newly declared variables from escaping.

The expression language allows you to refer to a *vari*able in scope, to the current instance of a class using *this* or to generate a *null* value. Given an expression, you can call a method in the corresponding class. For this you need to give a list of expressions typed in the same context. We represent a list in which all values share part of indices using *FVec*:

```
data FVec f xs where
  FNil  ::                          FVec f '[]
  FCons :: f x → FVec f xs → FVec f (x ': xs)
```

Finally, we need operations to weaken the environment. As hinted at in § 6, in this case it is not enough to remember only the first element in the environment, but it is necessary also to remove elements in between. For that reason the *weaken* combinator uses a *Remove* type-level operation.

The good news is that we only need to do this modelling of the language once. After that, we can add any number of extensions we want to the Java language.

8.2 Enhanced `for` for `Iterables`

Enhanced `for` loops were introduced in Java 5 to make it easier to iterate within collections. In short, a statement:

```
for (T x : e) s
```

is transformed into:

```
Iterator<T> it = x.getIterator();
while (it.hasNext()) { T x = it.next(); s }
```

The typing rule corresponding to this construct is:

$$\frac{\Gamma;\mathfrak{C} \vdash^{\mathcal{E}} e : \ell \qquad (x : \mathtt{T}, \Gamma);\mathfrak{C} \vdash^{\mathcal{S}} s : \omega \qquad \Gamma;\mathfrak{C} \vdash^{\mathcal{S}} r : \rho \quad \texttt{getIterator} \text{ is a method in } \ell \text{ returning } \texttt{Iterator<T>}}{\Gamma;\mathfrak{C} \vdash^{\mathcal{S}} \texttt{for (T } x \texttt{ : } e\texttt{) } s \texttt{ ; } r : \rho}$$

`Iterator<a>` is a given generic class in Java, which we need to encode in Haskell. Generics in Java correspond pretty nicely to type parameters in this representation.

```
data Iterator a
data HasNext
data Next
type instance JDirectDefinedMethods (Iterator a)
  = (HasNext, ([], Bool)) : (Next, ([], a)) : []
```

With this information in place we can write the type class representing this syntax extension:

```
data GetIterator
class Base wt ⇒ For wt where
  for :: JMethodSig ℓ GetIterator ∼ ([], Iterator α)
      ⇒ wt E 𝔈 Γ ℓ → wt S 𝔖 (α : Γ) ω
      → wt S 𝔖 Γ ρ → wt S 𝔖 Γ        ρ
```

Note that we can get away with demanding a *getIterator* method with the suitable signature to exist, we do not need the class to implement any specific interface. These restrictions are similar to those imposed in C# (Hejlsberg et al. 2003). The translation holds no surprises, except for the tricky manipulation of environments.

```
for expr inside rest
  = varDecl p⟨Iterator a⟩ $
    assign p⟨'Z⟩ (call (forget p⟨'Z⟩ expr)
                       p⟨GetIterator⟩ FNil) $
    while (call (var p⟨'Z⟩) p⟨HasNext⟩ FNil)
          (varDecl p⟨a⟩ $
           assign p⟨'Z⟩ (call (var p⟨'S 'Z⟩)
                              p⟨Next⟩ FNil) $
           forget p⟨'S 'Z⟩ inside) $
    forget p⟨'Z⟩ rest
```

8.3 Scala-like `for` Comprehensions

The Scala language includes an iteration construct very similar to list comprehensions. However, instead of translating down to list operations, it uses standard Java classes like `Iterator<a>`. We can implement this extension on top of the `for` loops we just described.

In order to keep the discussion short, we shall not distinguish between single qualifiers and sequences of qualifiers as we did for list comprehensions. Instead, we have a single data type which encodes the two possible combinators, patterns and guards, and expect a continuation for the comprehension. The body is given at the end of the sequence.

```
type family JDefinedMethods (ty :: JType) :: '[(JMethod, JSig)] where
  JDefinedMethods () = '[]   -- Stop
  JDefinedMethods ty = JDirectDefinedMethods ty ⊕ JDefinedMethods (JDirectSuperClass ty)
type family JMethodSig (ty :: JType) (name :: JMethod) :: JSig where
  JMethodSig ty name = Lookup name (JDefinedMethods ty)
type family Matches (sig :: [JType]) (actual :: [JType]) :: Constraint where
  Matches []       []       = ()
  Matches (e : ss) (e : as) = Matches ss as
  Matches (s : ss) (a : as) = (JSuperClass a s, Matches ss as)
```

Figure 6. Some definitions related to Java methods

```
type family Lookup (x :: v) (xs :: '[(k, v)]) :: k where
  Lookup k ((k, v) ': zs) = v
  Lookup k (z      ': zs) = Lookup k zs
type family (xs :: '[k]) ⊕ (ys :: '[k]) :: '[k] where
  '[]       ⊕ ys = ys
  (x ': xs) ⊕ ys = x ': xs ⊕ ys
type family Remove (n :: Nat) (xs :: '[k]) :: '[k] where
  Remove 'Z     (x ': xs) = xs
  Remove ('S n) (x ': xs) = x ': Remove n xs
```

Figure 7. Ancillary type families for type-level lists

```
data LangElt = S | E
class Base wt where
  -- Statements
  varDecl :: Proxy t
          → wt S C (t : Γ) r          -- rest of sequence with
                                      -- extended environment
          → wt S C Γ r
  assign  :: Ix i Γ ~ v ⇒ Proxy i → wt E C Γ v
          → wt S C Γ r                -- rest of the sequence
          → wt S C Γ r
  while   :: wt E C Γ Bool            -- iteration condition
          → wt S C Γ inner            -- body of while
          → wt S C Γ r → wt S C Γ r
  if_     :: wt E C Γ Bool →          -- condition
          → wt S C Γ inner            -- then part, no else
          → wt S C Γ r → wt S C Γ r
  bracket :: wt elt C Γ inner
          → wt S C vs r → wt S C vs r
  return_ :: wt E C Γ r → wt S C Γ r
  end     :: wt S C Γ r
  -- Expressions
  var     :: Proxy i → wt E C Γ (Ix i Γ)
  this    :: wt E C Γ C
  null    :: wt E C Γ e               -- null is in every class
  call    :: (JMethodSig e method ~ (params, ret)
             , Matches params ps)
          ⇒ wt E C Γ e → Proxy method
          → FVec (wt E C Γ) ps
          → wt E C Γ ret
  -- Weakening
  forget  :: Remove i Γ ~ Δ ⇒ Proxy i
          → wt elt C Δ r → wt elt C Γ r
```

Figure 8. Small Java language

```
data ForComprSyn wt C Γ where
  Pat    :: JMethodSig ℓ GetIterator ~ ([], Iterator a)
         ⇒ wt E C Γ ℓ      → ForComprSyn wt C (a : Γ)
         → ForComprSyn wt C Γ
  Guard  :: wt E C Γ Bool → ForComprSyn wt C Γ
         → ForComprSyn wt C Γ
  Final  :: wt S C Γ ω
         → ForComprSyn wt C Γ
```

For example, the following comprehension:

```
for (t <- tasks if t.isFinished)
  this.print(t.name)
```

is represented by the following Haskell value, where we omit the code to obtain *tasks* for conciseness:

```
Pat tasks $
  Guard (call (var p⟨'Z⟩) p⟨isFinished⟩ FNil) $
    Final (call this p⟨print⟩
                (FCons (call (var p⟨'Z⟩) p⟨name⟩ FNil) FNil))
```

The translation is very easy if we piggyback on the previous extension. Note that in the final case, we need to enclose the given statement in brackets to ensure that no variable leaves the environment.

```
class For wt ⇒ ForCompr wt where
  forcm :: ForComprSyn wt C Γ
        → wt S C Γ ρ → wt S C Γ ρ
  forcm (Pat    e es) r = for e (forcm es end) r
  forcm (Guard g es) r = if_ g (forcm es end) r
  forcm (Final  st)  r = bracket st r
```

Et voilà! Our tower of extensions is proven sound with respect to the typing rules of base Java. The example shows that technique applies to a broad range of languages, either entirely expression-based or two-stage language consisting of statements and expressions.

9. Related Work

Whereas this paper addresses the issue of checking for soundness of a translation to a host language using another external language (Haskell or Agda in this case), in many other systems the extensions are described and checked within the language itself. We have mentioned several of these systems in the introduction. However, there is a sharp distinction between two kinds of extension languages: those which ensure that the resulting code is always well-typed, and those which defer type checking until desugaring is done. We focus on the former, whose techniques relate to ours.

Typed Template Haskell (Peyton Jones 2010) and MetaOcaml (Kiselyov 2014) code which compiles is guaranteed to generate well-typed code. However, the resulting code cannot depend on the environment and typing information where it is going to live.

Furthermore, we are constrained to the type system of the host languages, Haskell or OCaml.

Our verification methodology targets the soundness problem as described for SoundX (Lorenzen and Erdweg 2016). Both systems are applicable to a variety of type systems. The approach, however, is quite different in several dimensions:

- SoundX uses a custom language to describe extensions and desugarings, whereas we express this information as a finally-encoded DSL in either Haskell or Agda. As a result, syntax in SoundX is more extensible, since it is not tied to a host language. We do not see this as a big problem, since we expect our verification technique to be used under the hood, where syntax is less of a problem.
- Both systems try to automatically prove soundness and generate translation of derivations. SoundX does so with a custom algorithm, whereas we piggyback on the host language type checker and elaborator. Our technique is thus simpler to implement and understand for those versed in the Haskell and Agda type systems; on the other hand we are at the mercy of those same compilers, and it can be difficult to understand why an extension fails to check.
- SoundX defines a class of languages extensions for which their algorithm is proven to work. In particular, there are some restrictions depending on whether a translation relies on its premises to be in desugared or non-desugared form. Our technique covers most of their problematic examples, like Church encoding of pairs or multiple **let** bindings, but we have not yet found a description of the extensions where our technique fails.
- Although we did not discuss our Agda implementation in this paper, When using Agda, we have a whole proof language at our disposal. That means that for those cases in which soundness cannot be readily checked, we can inform the compiler about the missing lemmas within the framework itself. This is quite important when induction is needed.

Acknowledgments

This work was supported by the Netherlands Organisation for Scientific Research (NWO) project on "DOMain Specific Type Error Diagnosis (DOMSTED)" (612.001.213).
We would like to thank PEPM reviewers for their insightful comments. Several sections were greatly improved by their suggestions.

References

R. Atkey, S. Lindley, and J. Yallop. Unembedding Domain-Specific Languages. In *Proceedings of the 2Nd ACM SIGPLAN Symposium on Haskell*, Haskell '09, pages 37–48. ACM, 2009.

M. Bolingbroke. Constraint Kinds for GHC, 2011. URL `http://blog.omega-prime.co.uk/?p=127`. Blog post.

E. Burmako. Scala Macros: Let Our Powers Combine!: On How Rich Syntax and Static Types Work with Metaprogramming. In *Proceedings of the 4th Workshop on Scala*, SCALA '13, pages 3:1–3:10. ACM, 2013.

J. Carette, O. Kiselyov, and C.-c. Shan. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *J. Funct. Program.*, 19(5):509–543, Sept. 2009.

B. Heeren, J. Hage, and S. D. Swierstra. Scripting the Type Inference Process. In *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming*, ICFP '03, pages 3–13. ACM, 2003.

A. Hejlsberg, S. Wiltamuth, and P. Golde. *C# Language Specification*. Addison-Wesley Longman Publishing Co., Inc., 2003.

M. V. Hermenegildo, F. Bueno, M. Carro, P. López-García, E. Mera, J. F. Morales, and G. Puebla. An Overview of Ciao and Its Design Philosophy. *Theory Pract. Log. Program.*, 12(1-2):219–252, Jan. 2012.

O. Kiselyov. Typed Tagless Final Interpreters. In *Proceedings of the 2010 International Spring School Conference on Generic and Indexed Programming*, SSGIP'10, pages 130–174. Springer-Verlag, 2012.

O. Kiselyov. The design and implementation of ber metaocaml, 2014.

F. Lorenzen and S. Erdweg. Modular and Automated Type-soundness Verification for Language Extensions. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, ICFP '13, pages 331–342. ACM, 2013.

F. Lorenzen and S. Erdweg. Sound Type-dependent Syntactic Language Extension. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL 2016, pages 204–216. ACM, 2016.

G. Mainland. Why It's Nice to Be Quoted: Quasiquoting for Haskell. In *Proceedings of the ACM SIGPLAN Workshop on Haskell Workshop*, Haskell '07, pages 73–82. ACM, 2007.

S. Marlow et al. Haskell 2010 Language Report. `https://www.haskell.org/onlinereport/haskell2010/`.

S. Peyton Jones. New directions for Template Haskell, 2010. URL `https://ghc.haskell.org/trac/ghc/wiki/TemplateHaskell/BlogPostChanges`.

F. Pfenning and C. Elliott. Higher-order Abstract Syntax. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation*, PLDI '88, pages 199–208. ACM, 1988.

T. Schrijvers, M. Sulzmann, S. Peyton Jones, and M. Chakravarty. Towards open type functions for haskell. In *19th International Symposium on Implemantation and Application of Functional Languages*, 2007.

A. Serrano and J. Hage. Type Error Diagnosis for Embedded DSLs by Two-Stage Specialized Type Rules. In *Programming Languages and Systems, ESOP 2016*, pages 672–698, 2016.

M. Sperber, R. K. Dybvig, M. Flatt, A. van Straaten, R. Findler, and J. Matthews. *Revised [6] Report on the Algorithmic Language Scheme*. Cambridge University Press, 1st edition, 2010.

S. Tobin-Hochstadt, V. St-Amour, R. Culpepper, M. Flatt, and M. Felleisen. Languages as Libraries. In *Proceedings of the 32Nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, pages 132–141. ACM, 2011.

T. Winant, D. Devriese, F. Piessens, and T. Schrijvers. Partial type signatures for haskell. In *Proceedings of the 16th International Symposium on Practical Aspects of Declarative Languages - Volume 8324*, PADL 2014, pages 17–32. Springer-Verlag New York, Inc., 2014. ISBN 978-3-319-04131-5.

B. A. Yorgey, S. Weirich, J. Cretin, S. Peyton Jones, D. Vytiniotis, and J. P. Magalhães. Giving Haskell a Promotion. TLDI '12, pages 53–66. ACM, 2012.